



PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL

FORMATO GUÍA DE APRENDIZAJE

1) IDENTIFICACIÓN DE LA GUIA DE APRENDIZAJE

- **Denominación del Programa de Formación:** Tecnología en Análisis y Desarrollo de Software
- **Código del Programa de Formación:** 3234206
- **Nombre del Proyecto:** Construcción de software integrador de tecnologías orientadas a servicios
- **Fase del Proyecto:** Ejecución
- **Actividad de Proyecto:** Introducción al control de flujo ciclos
- **Competencia:** DESARROLLAR LA SOLUCIÓN DE SOFTWARE DE ACUERDO CON EL DISEÑO Y METODOLOGÍAS DE DESARROLLO
- **Resultados de Aprendizaje Alcanzar:** Resolver procesos lógicos a través de la implementación de Algoritmos y el lenguaje de programación seleccionado.
- **Duración de la Guía:** 4 Horas 15 minutos

2) PRESENTACIÓN

Aprendiz SENA:

En el desarrollo de software, existen tareas que deben repetirse varias veces hasta lograr un resultado deseado. Resolver este tipo de situaciones de forma manual sería poco práctico y consumiría muchos recursos. Para atender esta necesidad, los lenguajes de programación incorporan estructuras denominadas **ciclos o bucles**, las cuales permiten ejecutar instrucciones de manera repetitiva bajo ciertas condiciones.

El estudio de los ciclos constituye un paso fundamental en la formación de un programador, pues facilita la construcción de soluciones más eficientes, ordenadas y escalables. A través de este tema, los aprendices descubrirán cómo controlar la repetición de procesos en sus programas, aplicando lógica estructurada y fortaleciendo sus habilidades de pensamiento computacional.



3) FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

A. Actividades de reflexión inicial.

En nuestra vida diaria realizamos acciones que se repiten constantemente: cepillarnos los dientes, subir las escaleras, saludar a las personas, preparar café o tender la cama. Estas tareas siguen un patrón que se repite con cierta frecuencia, lo que nos permite ahorrar tiempo y energía porque no tenemos que inventarlas desde cero cada día.

De manera similar, en programación necesitamos resolver problemas que requieren repetir instrucciones varias veces, bajo una condición específica o hasta cumplir un objetivo. Los **ciclos en programación** son la herramienta que nos permite lograrlo, ofreciendo eficiencia y claridad en la construcción de algoritmos.

Actividades de aprendizaje:

Enunciado:

Imagina que tienes que llenar una botella de agua con un vaso pequeño. Para lograrlo, debes repetir una misma acción: tomar agua con el vaso y vaciarla dentro de la botella. Si tuvieras que escribir cada uno de esos pasos de manera individual, tu lista sería larguísima. Sin embargo, si pudieras indicar que ese proceso debe repetirse hasta que la botella esté llena, simplificarías mucho la tarea.

Preguntas para reflexionar:

1. ¿Qué otras acciones realizas en tu vida diaria que requieren repetir un mismo paso varias veces hasta cumplir una meta (por ejemplo, barrer, correr vueltas a la cancha, tender varias camas)?
2. ¿Qué ventajas tiene pensar en estas acciones como repeticiones automáticas y no como instrucciones que debes escribir una y otra vez?
3. ¿Cómo relacionarías este tipo de situaciones con lo que hace un programador cuando necesita que el computador repita ciertas instrucciones?

Estrategias didácticas activas:

- Analogía con la vida diaria



- Lluvia de ideas
- Discusión guiada
- Aprendizaje colaborativo

Duración de la actividad 15 minutos.

B. Actividades de contextualización e identificación de conocimientos necesarios para el aprendizaje.

En la programación, los **ciclos** nos permiten repetir un conjunto de instrucciones sin necesidad de escribirlas una y otra vez. Para comprenderlo mejor, retomemos ejemplos cotidianos: al correr varias vueltas en una pista, al servir varias tazas de café en una reunión o al realizar varias sumas en una lista de compras, repetimos acciones similares hasta lograr un objetivo.

En el desarrollo de software sucede lo mismo: un programador puede necesitar que un computador repita cálculos, recorra una lista de datos o muestre mensajes múltiples veces. Para resolverlo de manera eficiente, utiliza **estructuras de control repetitivas** (ciclos).

Actividades de aprendizaje:

Ejercicio 1:

Construye un diagrama de flujo que muestre los números del 1 al 10, pero que solo imprima los números pares.

(Pista: usa un ciclo con una condición que evalúe si el número es par).

Ejercicio 2:

Dibuja un diagrama de flujo que represente a un estudiante que repasa una tabla de multiplicar 5 veces, pero que puede detenerse antes si obtiene un resultado incorrecto.

(Pista: incluye un ciclo y dentro de él una decisión que valide la respuesta).



Ejercicio 3:

Crea un diagrama de flujo para repartir dulces a 10 niños. Si un niño ya tiene un dulce, sáltalo y continúa con el siguiente hasta terminar.

(Pista: el ciclo representa la entrega y la condición verifica si el niño ya tiene dulce).

Materiales de formación:

Estrategias didácticas activas:

- Clase magistral breve con mediación activa.
- Aprendizaje activo
- Aprendizaje significativo

Duración de la actividad 30 minutos.

C. Actividades de apropiación del conocimiento.

resolver problemas reales y de **transformar nuestra manera de pensar**. Según la pedagogía de Paulo Freire, el conocimiento se construye en diálogo con la vida cotidiana, por lo que cada ejercicio debe permitirte reflexionar, cuestionar y aplicar lo aprendido.

En este momento, trabajarás con **ciclos y condicionales** a través de diagramas de flujo. El objetivo es que no solo logres repetir instrucciones, sino que seas capaz de tomar decisiones dentro de esas repeticiones, tal como sucede en la vida diaria: no siempre actuamos igual, muchas veces decidimos en función de lo que ocurre a nuestro alrededor.

Instrucciones

Desarrolla los siguientes ejercicios en diagramas de flujo:

1. Diseña un diagrama de flujo que muestre los números del 1 al 20, pero solo imprima los múltiplos de 3.



2. Construye un diagrama de flujo que permita registrar el ingreso de 5 estudiantes a un salón. Si un estudiante no trae su carné, no debe ingresar y el ciclo debe continuar con el siguiente.
3. Realiza un diagrama de flujo que pida al usuario ingresar 10 números y muestre únicamente los que sean mayores que 50.
4. Diseña un diagrama de flujo que simule un cajero automático: repite el proceso de retirar dinero hasta que el saldo sea insuficiente.
5. Elabora un diagrama de flujo que represente una competencia de atletismo con 8 corredores. Cada corredor da 4 vueltas, pero si alguno se cansa antes de terminar, se detiene y el ciclo continúa con los demás.
6. Construye un diagrama de flujo que simule la venta de 15 boletos de cine. Si el comprador es menor de edad y la película es para adultos, se debe rechazar la venta y continuar con el siguiente comprador.
7. Diseña un diagrama de flujo que sume los números del 1 al 100, pero interrumpa el ciclo si encuentra un número mayor a 90 que sea par.
8. Crea un diagrama de flujo que lea el nombre y la edad de 5 personas. Solo mostrará en pantalla aquellas personas que tengan 18 años o más.

Materiales de formación:

- Documento Word

Estrategias didácticas activas:

- Aprendizaje activo:
- Resolución de problemas prácticos
- Trabajo autónomo guiado
- Retroalimentación entre pares

Evidencias de aprendizaje:



- Documento Word con las respuestas

Duración de la actividad 1 hora 30 minutos.

D. Actividades de transferencia del conocimiento.

Has recorrido un camino importante: aprendiste a declarar variables y constantes, a usar operadores matemáticos para realizar cálculos, a aplicar condicionales para tomar decisiones y, finalmente, a usar ciclos para repetir acciones. Ahora es momento de **integrar todos estos conocimientos en problemas prácticos**, de manera que demuestres tu capacidad para construir soluciones completas.

Actividades de aprendizaje:

Ejercicios Prácticos

Ejercicio 1 – Números primos

Crea un programa que: muestre todos los números primos entre 1 y 50. Utiliza ciclos y condicionales para verificar qué números cumplen la condición de ser primos.

Ejercicio 2 – Tabla de multiplicar personalizada

Crea un programa que: dado un número ingresado por el usuario, muestre su tabla de multiplicar hasta el 12. El ciclo debe permitir repetir el proceso si el usuario desea consultar otra tabla.

Ejercicio 3 – Cajero automático

Crea un programa que: simule un cajero automático. El usuario inicia con un saldo definido en una variable constante. Podrá retirar dinero varias veces mientras tenga fondos suficientes. Si intenta retirar más de lo que tiene, debe aparecer un mensaje de error y no permitir la transacción.

Ejercicio 4 – Promedio de notas

Crea un programa que: lea las notas de 5 estudiantes. Calcula el promedio de cada uno y



determina, mediante condicionales, si aprueba (nota ≥ 3.0) o reprueba. Usa ciclos para ingresar los datos de manera ordenada.

Ejercicio 5 – Tienda de productos

Crea un programa que: simule la venta de productos. El usuario podrá registrar la compra de varios artículos hasta que decida no comprar más. Se debe calcular el total de la compra y aplicar un descuento del 10% si el valor supera los \$100.000.

Ejercicio 6 – Juego del adivinador

Crea un programa que: genere un número aleatorio entre 1 y 20. El usuario intentará adivinarlo y el ciclo repetirá los intentos hasta que lo logre. Cada vez que falle, el programa debe indicar si el número buscado es mayor o menor que el ingresado.

Ejercicio 7 – Contador de pares e impares

Crea un programa que: permita al usuario ingresar 10 números. Al final, muestra cuántos fueron pares y cuántos impares.

Ejercicio 8 – Control de acceso

Crea un programa que: solicite un usuario y contraseña. El programa debe permitir hasta 3 intentos; si el usuario falla en todos, mostrar un mensaje de “Acceso denegado”. Si acierta, mostrar “Bienvenido”.

Materiales de formación:

- Editor de código (VS Code)
- Navegador web con herramientas de inspección
- Repositorio de Git

Estrategias didácticas activas:

- Resolución de problemas prácticos.
- Aprendizaje significativo en contextos cotidianos.



- Trabajo autónomo con retroalimentación del instructor.
- Uso de GitHub como portafolio de evidencias.

Evidencias de aprendizaje:

- La solución de los ejercicios en archivos separados
- Comentarios en el código explicando el uso de condicionales.
- Registro de commits claros en GitHub.
- Comentarios de los análisis requeridos

Duración de la actividad 2 horas.

4) ACTIVIDADES DE EVALUACIÓN

Tome como referencia la técnica e instrumentos de evaluación citados en la guía de Desarrollo Curricular

Fase del proyecto formativo	Actividad del proyecto formativo	Actividad de Aprendizaje	Evidencias de Aprendizaje	Criterios de Evaluación	Técnicas e Instrumentos de Evaluación
EJECUCIÓN	DESARROLLAR LA ESTRUCTURA DE DATOS, CONSTRUCCIÓN, CODIFICACIÓN E INTERFAZ DE USUARIO, PRUEBAS Y VALIDACIÓN DE LOS MÓDULOS DEL SISTEMA DE INFORMACIÓN	Identifica y describe, en un sistema de información dado, los datos de entrada, procesamiento de los datos e información generada, según necesidades del cliente.	Conocimiento y Producto	Solucionar problemas de lógica proposicional. Usar estructuras secuenciales en la construcción de algoritmos. Utilizar estructuras de control en la construcción	Listas de chequeo



				de algoritmos. Emplear estructuras cíclicas en la construcción de algoritmos. Construir algoritmos con arreglos.	
--	--	--	--	---	--

5) GLOSARIO DE TÉRMINOS

- **Algoritmo:** Conjunto de pasos ordenados y finitos que permiten resolver un problema o cumplir una tarea en programación.
- **Bucle (Ciclo):** Estructura de control que permite repetir una o varias instrucciones mientras se cumpla una condición específica o un número definido de veces.
- **Condición:** Expresión lógica que puede ser verdadera o falsa y que determina el flujo de ejecución en condicionales y ciclos.
- **Condicional:** Estructura que permite tomar decisiones en un programa, ejecutando un bloque de instrucciones solo si se cumple cierta condición.
- **Ciclo “mientras” (While):** Tipo de ciclo que repite instrucciones mientras la condición sea verdadera. Evalúa la condición antes de ejecutar el bloque de código.
- **Ciclo “hacer mientras” (Do While):** Ciclo que ejecuta al menos una vez las instrucciones y luego evalúa la condición para determinar si debe repetirse.
- **Ciclo “para” (For):** Ciclo que repite instrucciones un número determinado de veces, generalmente controlado por un contador.



- **Contador:** Variable que se incrementa o decrementa en cada repetición de un ciclo y que se utiliza para llevar control de las iteraciones.
- **Iteración:** Cada una de las repeticiones que realiza un ciclo.
- **Operadores matemáticos:** Símbolos que permiten realizar operaciones aritméticas como suma (+), resta (-), multiplicación (*), división (/), o módulo (%).
- **Operadores lógicos:** Símbolos que permiten combinar condiciones, como AND (&&), OR (||) y NOT (!).
- **Variable:** Espacio en memoria donde se almacena un dato que puede cambiar durante la ejecución del programa.
- **Constante:** Espacio en memoria que almacena un valor que no puede modificarse durante la ejecución del programa.
- **Condiciona anidada:** Estructura en la que una condición se encuentra dentro de otra, permitiendo decisiones más complejas.
- **Diagrama de flujo:** Representación gráfica de un algoritmo mediante símbolos y flechas que muestran el flujo de las operaciones, decisiones y ciclos.

6) REFERENTES BIBLIOGRÁFICOS

- Deitel, P., & Deitel, H. (2019). *Cómo programar en Java* (11.^a ed.). Pearson Educación.
- Joyanes Aguilar, L. (2017). *Fundamentos de programación: Algoritmos, estructuras de datos y objetos*. McGraw-Hill.
- Malvino, A. P., & Brown, J. A. (2018). *Principios de programación estructurada*. McGraw-Hill.
- Pressman, R. (2021). *Ingeniería de software: Un enfoque práctico* (9.^a ed.). McGraw-Hill.
- Forbellone, A. L., & Eberspächer, H. F. (2000). *Lógica de programación: La construcción de algoritmos y estructuras de datos*. Prentice Hall.
- Freire, P. (2005). *Pedagogía del oprimido*. Siglo XXI Editores.
- SENA (2022). *Guía metodológica para la elaboración de guías de aprendizaje*. Servicio Nacional de Aprendizaje – SENA.



- W3Schools. (s.f.). *JavaScript Loops*. Recuperado de: https://www.w3schools.com/js/js_loop_for.asp
- GeeksforGeeks. (s.f.). *Loops in Programming Languages*. Recuperado de: <https://www.geeksforgeeks.org/loops-in-programming-languages>

7) CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	John Freddy Becerra Castellanos	Instructor Contratista	CIMI	09/29/2025

8) CONTROL DE CAMBIOS (diligenciar únicamente si realiza ajustes a la guía)

	Nombre	Cargo	Dependencia	Fecha	Razón del Cambio
Autor (es)					